

Lecture 5: Unicode & bitwise operations

CMPE 120: Computer Organization & Architecture

Olivia Weng

What about non-English characters?

What about non-English characters?

- Thousands more characters used in languages around the world
- ASCII does not define:
 - Spanish: é
 - Chinese: 中
 - Emoji: 🐪 ←
- char datatype of 1 byte only encodes 256 possible bit patterns
- Challenge: Millions of lines of code written that assumed 1 byte ASCII chars

UTF-8: Unicode encoding

- Use more bits to encode more characters!

- Code point: an integer representing a character (e.g., 'A' == 65)

- Normal ASCII code point: Highest order bit of byte is 0xxxxxxx

- UTF-8 is backwards compatible with ASCII!

- Multi-byte code point: Highest order bit of byte is 1xxxxxxx

'A' == 65

MSB

0-127

MSB

$$x = 10r0$$

How many bytes do you need?

- Multi-byte code point: Highest order bit of byte is 1xxxxxxx

- Bit flags indicate code point length

○ 110xxxxx = 2 bytes

○ 1110xxxx = 3 bytes

○ 11110xxx = 4 bytes

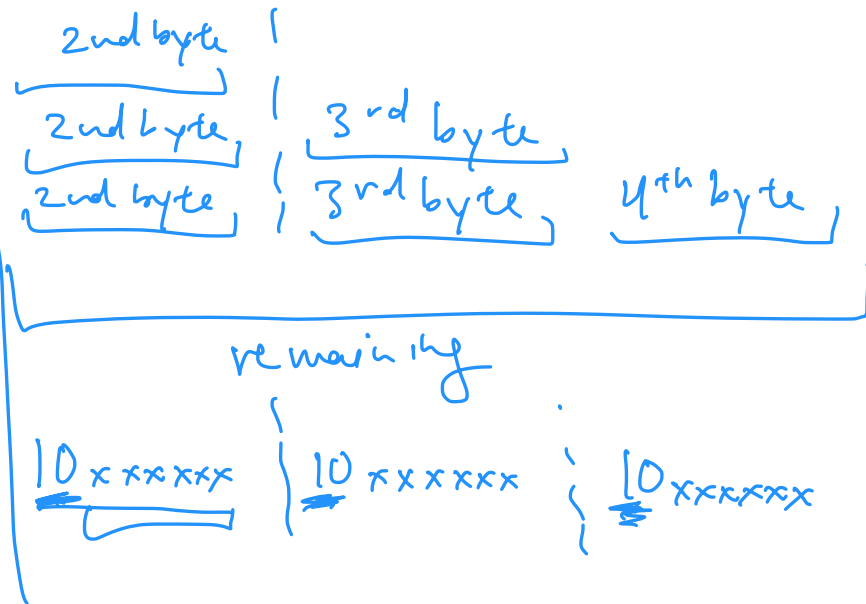
- Bytes after the first byte start with

○ 10xxxxxx

110xxxxx

1110xxxx

11110xxx



San José

Code point construction

2-byte code point

- $\text{é} = 11000011\ 10101001$
 - bit flags : only encodes metadata (i.e., only provides information about)
 - the code point = $000011\ 101001 = 233$
 - this is a byte following the first byte
 - tells us this is 2 bytes long

233 $\Rightarrow$ é

Demo

```
char str[] = "José";
```

```
for (int i = 0; str[i] != '\0'; i++) {  
    printf("0x%x\n", (unsigned char) str[i]);  
}
```

How many bytes do you need?

- **Multi-byte** code point: Highest order bit of byte is **1xxxxxxx**
- **Bit flags** indicate code point length
 - 110xxxxx = 2 bytes
 - 1110xxxx = 3 bytes
 - 11110xxx = 4 bytes
- Bytes after the first byte start with
 - 10xxxxxx
- How do we check for these specific bit flags to know how many bytes we have?
 - Need a way of inspecting individual bits!

Bit operations

- Special mathematical operations in C for manipulating bits
 - `&`: AND
 - `|`: OR
 - `~`: NOT
 - `^`: XOR
 - `>>`: shift right
 - `<<`: shift left
- 

AND: &

$a \& b = ?$

\$\$

- AND each bit together

Truth table

input_a	input_b	Result
0	0	0
0	1	0
1	0	0
1	1	1

1010 1100
↓ ↓ ↓ ↓
& 0000 1111

0000 1100

What is the result?

$$\begin{array}{r} \boxed{\&} \quad \begin{array}{r} 00110011 \\ 10100101 \end{array} \\ \hline 00100011 \end{array}$$

OR: |

- OR each bit together

Truth table

input_a	input_b	Result
0	0	0
0	1	1
1	0	1
1	1	1

$$\begin{array}{r} 11001100 \\ | 00001111 \\ \hline 11001111 \end{array}$$

What is the result?

$$\begin{array}{r} 00100111 \\ 10101100 \\ \hline 10101111 \end{array}$$

A handwritten blue '1' is circled and underlined, indicating a carry-in of 1 to the leftmost column of the addition.

NOT: ~

1 operand

- NOT each bit together

Truth table

input_a	Result
0	1
1	0

~ 1111 0000

0000 1111

Shift right: $\gg$

- Shift the bits "off a cliff" to the right

- Logical shift
 - Zero extension
 - Used to shift unsigned values
- Arithmetic shift:
 - Sign extension
 - Used to shift signed values

MSB: 0 $\rightarrow$ extend w/ 0-bit

1 $\rightarrow$ extend w/ 1-bit

logical shifting

$$0011 \gg 1 = 0001$$

$$\rightarrow 0001 \times$$

$$0011 \gg 2 = 0000$$

$$\rightarrow 00 \times \times$$

$$100 \times \gg 1$$

$$= 0100$$

Arithmetic shift

$$001 \times \gg 1 = 0001$$

$$1100 \times \gg 1 = 1100$$

4-bit signed

What is the result in binary and decimal?

-6

• $1010 \gg 3 = 1111 (-1)$
↑
 $-6 \div (3 \times 2) = -1$

$1010 \gg 1 = 1101 (-3)$
↑
%.2

$1101 \gg 1 = 1110 (-2)$
↑
%.2

• $0100 \gg 1 = 0010 (2)$
↑
%.2

• $111011 \gg 4 = 1111 (-1)$
↑
 $(-5) \div (4 \times 2) = -1$

• $11101 \gg 2 = 1111 (-1)$
↑
 $(-2) \div (2 \times 2) = -1$

Shift left: <<

- Shift the bits "off a cliff" to the left

1100 << 1

*100 ← = 1000

4-bit unsigned

What is the result in binary and decimal?

- $\overset{(2)}{0010} \ll 2 = 1000 \text{ (8)}$
 $2 \times (2^2) = 8$
- $\overset{(1)}{0001} \ll 3 = 1000 \text{ (8)}$
 $1 \times (2^3) = 8$
- $\overset{(8)}{1000} \ll 4 = 0000 \text{ (0)}$
 $8 \times (2^4)$
- $\underline{0110} \ll 2 = 1000$

Demo: practice with bitwise operators

& | >> << ~

int bitwise_is_even(int8_t num);

return 1 if num is even else 0

int count_1_bits(int32_t num);

count of 1s

110 —
1110 —
11110 —
└────────┘

1st byte multi-byte
codepoint

How to use bit operators to select specific bits? ^{UTF-8}

How to use bit operators to select specific bits?

- Bit masking: select specific individual bits out of a binary representation of a number
- Examples:

◦ $\text{lowest_four_bits}(0b01010101) =$

$$\begin{array}{r} 0x00001111 \\ \hline 00000101 \end{array}$$

Handwritten: 00000101

◦ $\text{highest_four_bits}(0b10110011) =$

$$\begin{array}{r} 0x11110000 \\ \hline 10110000 \end{array}$$

Handwritten: 10110000

◦ $\text{lowest_four_bits}(192) =$

$$192 - 128 = 64$$

Handwritten: $11000000 = 00000000$

◦ $\text{highest_four_bits}(\text{lowest_four_bits}(200)) =$

$$200 - 128 = 72 - 64 = 8$$

Handwritten: $11001000 = 00001000 = 00000000$

How to implement masking?

```
char lowest_four_bits(char c) {  
    return c & 0b00001111;  
}
```

bit mask

```
char highest_four_bits(char c) {  
    return c & 0b11110000;  
}
```

bit mask

Why can't we use the bitwise | operator?

The bitwise & operator selects specific bit positions based on the pattern of 1's that the variable is &d with.

& truth table	Result
0 & 0	0
0 & 1	0
1 & 0	0
1 & 1	1

How do we check the codepoint bit flags?

- Bit masking!
- `uint8_t codepoint_size(char string[])`
 - `// return the number of bytes this codepoint is, e.g., 1, 2, 3, or 4`

Demo

11 0 xxxxx 10 xxxxxx

xxxxxxxxxx = 233 in binary

- `int32_t code_point_2(char c1, char c2);` // return codepoint
 - é = 233

- `uint32_t utf8_strlen(char string[]);`

// Given a `char[]` representing a UTF-8 encoded string,

// return the number of UTF-8 codepoints (characters) in the string.

// The input will always be valid UTF-8 and will not exceed 2048 bytes.